

ALVEARE: a Domain-Specific Framework for Regular Expressions

Filippo Carloni
filippo.carloni@polimi.it
Politecnico di Milano
Milano, Italy

Davide Conficconi
davide.conficconi@polimi.it
Politecnico di Milano
Milano, Italy

Marco D. Santambrogio
marco.santambrogio@polimi.it
Politecnico di Milano
Milano, Italy

ABSTRACT

Regular Expression (RE) matching enables the identification of patterns in datastreams of heterogeneous fields ranging from proteomics to computer security. These scenarios require massive data analysis that, combined with the high data dependency of the REs, leads to long computational times and high energy consumption. Currently, RE engines rely on either (1) flexibility in run-time RE changes and broad operators support impairing performance or (2) fixed high-performing accelerators implementing few simple RE operators. To overcome these limitations, we propose ALVEARE: a hardware-software approach combining a Domain-Specific Language (DSL) with an embedded Domain-Specific Architecture. We exploit REs as a DSL by translating them into flexible executables through our RISC-based Instruction Set Architecture that expresses from simple to advanced primitives. Then, we design a speculation-based microarchitecture to execute real benchmarks efficiently. ALVEARE provides RE-domain flexibility and broad operators' support and achieves up to 34× speedup and 57× energy efficiency improvements against the state-of-the-art RE2 and Bluefield DPU 2 with its RE accelerator.

KEYWORDS

Regular Expressions, Domain-Specific Architecture

ACM Reference Format:

Filippo Carloni, Davide Conficconi, and Marco D. Santambrogio. 2024. ALVEARE: a Domain-Specific Framework for Regular Expressions. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3657378>

1 INTRODUCTION

Pattern-matching based on Regular Expressions (REs) finds relevant applications in various scenarios such as text analytics [15], computer security [3, 7, 26], and proteomics [18]. However, these contexts demand massive data analysis, which, combined with the high data dependency of the REs features, can quickly result in long computational times and high energy consumption [15, 16]. While traditional REs represent patterns through alternations and iterations, exceeding simple string matching, modern REs exhibit advanced features demanding more powerful primitives than the simpler ones [21]. For instance, character classes (e.g., `[A-Z]`) require

alternations among all the characters while bounded repetitions are transformed into unfolded sequences of concatenations (e.g., `A{3,5}`). Such transformation results in inefficient performance scaling of the matching engines [11], making their management challenging and demanding for advanced primitives.

Currently, the State of the Art split into Software (SW)-based, automata-acceleration-based, and Domain-Specific Architecture (DSA)-based. The first one supports the most advanced operators thanks to the flexibility of CPUs [10, 24]. However, low-latency systems require near-to-data analysis, usually exploiting embedded processors, to avoid costly data transfers to the central system, e.g., Packet Inspection on SmartNICs, excluding mainstream CPUs [8]. Similarly, other approaches save precious CPU cycles through GPU offloading [12], risking missing latency requirements. The second class of approaches embeds automata representations into either in-memory [5, 19] solutions or FPGA-based ones [17]. However, they struggle to handle commonly used advanced RE operators, such as in PCRE standard, or even fail in their support. Lastly, the DSA-based approach combines the benefits of the two previous ones with SW flexibility and Hardware (HW) specialization [2]. Nevertheless, they support a limited number of primitives [4, 14].

Based on these observations, we propose ALVEARE: a **full-stack domain-specific framework** to overcome the state-of-the-art limitations and target constrained and near-data scenarios where mainstream CPUs are either unavailable or saving their cycles is paramount. Using the REs as a Domain-Specific Language (DSL), we design a **novel domain-specialized RISC-based Instruction Set Architecture (ISA)** tailored to compactly express REs most common operators. We then combine the **flexibility of a compilation flow** with the efficiency of HW approaches through a **specialized DSA**. We prototype our DSA on an embedded FPGA and assess its execution time and energy efficiency against state-of-the-art solutions in different scenarios. Overall, ALVEARE achieves up to 34× speedup and 57× energy efficiency improvement against state-of-the-art solutions for constrained and near-data scenarios, while overwhelming GPU-based one with a minimum speedup of 356×. To summarize, we make a three-fold contribution:

- A **RE-tailored** domain-specialized **ISA** that support simple and advanced REs primitives for compact and efficient REs representation (§4);
- A **full-stack HW/SW methodology** relying on a **flexible and optimized compiler** moving the complexity from the hardware to the software, strongly simplifying the hardware design (§5) and on a **specialized microarchitecture** for the efficient execution of REs (§6).
- A **hardware mechanism** to handle speculative counter modes (greedy and lazy) supporting different matching modalities employed in real applications (§6).

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
DAC '24, June 23–27, 2024, San Francisco, CA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0601-1/24/06.
<https://doi.org/10.1145/3649329.3657378>

2 RELATED WORK

Current REs matching solutions aims to combine **efficiency** and **flexibility**. Especially, they split based on the supported *operators*, way to handle *dependencies*, if leveraging *NFA* and *DFA* models, or the employment of *enumerative* and *speculative* approaches. A key feature of these engines is the **execution architecture** and their **software stack** to make them practically usable.

CPU-based solutions generally present the higher degree of flexibility [10] and try to squeeze out the performance from the architectural characteristics [24] or exploit algorithmic improvements for dynamic adaptation among speculation or enumeration [15, 16]. Differently, **GPU-based solutions** exploit the SIMD/SIMT capabilities for multi-FSM and multi-striding [12], but the *embarrassingly sequential* [15] nature of this kernel highly limits their adoption [13].

Other solutions exploit **FPGAs** [17], **ASICs** [9], or **In-Memory** [5, 19, 20] **accelerators** to directly embed their NFA formulation in the reconfigurable fabric of the given architecture. Among the solutions for logic embedding, FPGAs represent the commercially available platform with higher potential [13]. However, these approaches suffer the lack of advanced primitives [11] and the long reconfigurability issue as for other In-Memory architectures [13, 19, 25].

To fill the gap among flexible and highly specialized solutions **software-programmable DSAs** are a promising approach that transforms patterns in sequences of instructions to execute [2, 4, 6, 14, 22]. However, complex REs primitives are poorly considered, and the main target scenario are datacenters. Among them, the NVIDIA Bluefield DPU 2 [2] is a SmartNIC with a RE accelerator to offload the pattern matching task that avoid CPU cycles burning.

Our methodology falls in these RE-centric architectures. However, our DSA combines a full-stack HW/SW approach to bring REs as first-class citizens covering a wide range of operators.

3 ALVEARE RE-CENTRIC FRAMEWORK

Our framework¹ leverages a flexible RISC-based ISA that decomposes REs in a set of primitives (§4) to support the widely adopted standards POSIX ERE and PCRE. The ISA breaks down into primitives performed intra-characters (e.g., concatenation or alternation) and among sub-RE (e.g., counters) accounting for dependencies and parallelization opportunities. The software-centric component of ALVEARE framework is the DSL compilation flow (§5). Its flexibility allows lifting part of the REs complexity towards the compiler to simplify the ISA and microarchitecture designs. Finally, the microarchitecture performs vector RE matching on the stream of data, while a *tailored controller* handles the data dependency and control issue through a depth-first-like speculation approach with a backtracking mechanism for *mispredictions* (§6).

4 RE-TAILORED ISA

The ISA relies on an operators-composition strategy similar to a VLIW approach to build advanced instructions, but with a fixed size. As Figure 1 illustrates, each instruction is 43 bits wide and breaks down into three fields: *opcode*, *enabling character*, and *reference*.

Bits 42 to 36 encode the operation; an operator is enabled when the corresponding bit is set to “1”. The default operation among instructions is the AND, meaning that the match process is complete

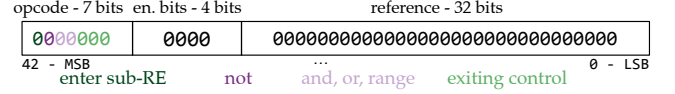


Figure 1: Our 43-bit instruction format breaks into three fields: 7-bit for the opcode (Table 1), 4 bits to validate the following 4 bytes, and 32 bits for the reference. E.g., ([^A-Z])+ translates second and third field of the second instruction as “1100” and “01000001 01011010” (the other bits are zeros).

Table 1: ALVEARE operation classes. Some of these operations are composable intra-class (e.g., the NOT) or inter-class (e.g.,) or QUANT with base), via the *don’t care* terms (-). E.g., ([^A-Z])+ translates “1000000”, “0111010”, and “0000000”.

Class	Operator	Opcode	Description
Control	EoR	0000000	End of RE
Base	AND	0010---	Char-based And
	OR	0001---	Char-based Or
	RANGE	0011---	Char-based Range
	NOT	01-----	Match Inversion
Complex	(	1000000	New Sub-RE
	)	0---100	End of Sub-RE
	QUANT L	0---001	) + Lazy Quantifier
	QUANT	0---010	) + Greedy Quantifier
	)	0---011	) + OR of Sub-RE

when all the instructions of the RE have a *sub-match*. Notably, to modify this behavior, we use advanced operators such as complex OR or quantified repetitions to force the execution flow to skip or repeat instructions. The bits from 35 to 32 are reference-enabling bits, *base operators* use them to identify whether a character in the reference is valid by setting them to “1”. Specifically, the reference field encodes sequentially the 8-bit characters pattern for base operators, or it encodes execution information such as the number of repetitions or the instructions relative addresses to jump for the complex operators. Given their sequential encoding, the enabling bits are “0” ended (e.g., A|B is represented as an OR operation with “1100” as enabling bits and “AB” as reference). Such reference-enabling bits are essential in binary-based pattern-matching applications [3], where we also need not human readable ASCII values (e.g., \x00).

Table 1 shows the three operator classes in our ISA: *control*, *base*, and *complex*, which handle the execution end, the intra-characters operations, and intra-REs operations, respectively. Notably, the ISA allows operators from different classes to be active in the same instruction if and only if only one of them uses the reference part.

ISA Simple REs: Control and Base Operators. The *control operator* instruction class comprises only the End of RE (EoR) operator, indicating the end of the RE matching process.

The *base operator* class represents all the basic intra-characters operations with their execution primitives. These operators have a one-to-one correspondence with REs DSL and can be used only between strings of characters. While the AND and OR are relatively intuitive, the RANGE represents the first optimization step. In principle, a RANGE operation is a sequence of OR alternations among

¹<https://github.com/necst/alveare>

a set of characters, reporting a match if one of them has a correspondence in the analyzed data. For instance, all the lowercase alphabetical characters [a-z] are also representable as a OR b ... OR z. However, such a big chain of alternations implies many instructions. Thus, we design an ISA RANGE operation primitive to heavily reduce the instruction count and the execution cycles (see §7.1). The NOT is another advanced primitive that exploits the ISA's composable structure and thus negates alternation operations (i.e., OR and RANGE) for PCRE and POSIX standards (Table 1).

The base operator class encodes the characters to match into the 32-bits of the reference field for at most four 8-bit vectorized characters. While the AND operator requires a correspondence among instruction and data characters, the OR requires a single character match. Similarly, the RANGE matches if the processed data contains a character within the start and the end of at least one of the two RANGES. The ISA enables the packing of [a-z0-9] in a single RANGE.

ISA Advanced REs: Complex Operators. The *entering sub-RE operator*, indicated as (, represents the entry point for a sub-pattern in the RE flow. Figure 2 shows the structure of its reference to set flags and information, hinting at the sub-RE execution type. The first five bits enable a valid reference subfield containing other helpful execution information. The two leftmost bits enable the counting repetition limits, i.e., a possible minimum and maximum, to express bounded quantifiers or Kleene's operators. We optimize the ISA design with a single counter primitive to be bounded or not in the sub-RE repetitions. We encode an infinite upper bound with all the bits set to "1". Thus, as the minimum and maximum counters encode with 6 bits, the supported range of the bounded counter is between 0 and 62 ($2^6 - 2$), and 63 is reserved for the unbounded.

The following 2 bits in the complex operator field (the 3rd and 4th) indicate whether the backward and forward addresses are valid (Figure 2). Specifically, the execution flow of an RE is extremely data-dependent; therefore, we might shift from the sequential execution flow and jump forward or backward in our RE based on the matching outcome. Hence, these backward and forward addresses specify the relative jumps to the address of the *entering sub-RE operator*, limiting the code size and improving the performance. The last of the five enabling bits sets and anticipates if the sub-RE execution is *greedy* or *lazy* (by setting the bit to "1").

The remaining operators of the complex class conclude the sub-RE execution in three different ways. The *quantification* repeats the sub-RE instructions zero or more times based on the specification at the entrance of the sub-RE. The execution flow jumps backward, thus repeating the sub-RE, or forward, hence continuing with the next operation. Notably, until the minimum number of repetitions is not satisfied, the execution can only jump backward. Instead, if the minimum limit is reached, the greedy or lazy mode impacts the execution. Specifically, the greedy mode repeats the sub-RE, thus jumping backward as many times as possible until a mismatch occurs. Differently, the lazy tries to move to the operation that follows the quantified sub-RE, repeating the sub-RE match only in case of a mismatch for the following operator. The *alternation of sub-REs* (i.e., |) jumps to the end of the alternation chain (forward jump) if the analyzed sub-pattern has a match. In contrast, if the sub-pattern has a mismatch, it jumps to the following sub-RE in the alternation chain (backward jump). Finally, the *simple termination*

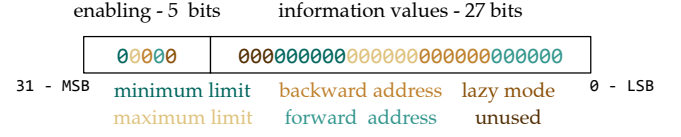


Figure 2: Reference detail for the enter sub-RE operator. Five bits are enabler bits for the following 27-bit validity of the respective subfields or to enable lazy or greed matching. The 27 bits encode the counter limits and the backward and forward relative jumps, while the 3 MSBs are currently unused. E.g., ([^A-Z])+ translates the first instruction as “1110” “00000000111111100000000000000000”.

of a sub-RE notifies a sub-pattern conclusion. Since all the *closing complex* operators do not use the reference, they are combined with the base operators to reduce the instructions count.

5 COMPILATION FLOW

Our compiler comprises the three traditional stages that transform a RE down to the DSA executable: front-end, mid-end, and back-end.

Front-End: Lexical and Syntax Analysis. The front-end takes the input REs and checks their lexical and syntactic compliance. The lexer employs *FLEX* tool and provides specific rules for tokenizing the supported RE operators. Then, we build a grammar of supported REs via BISON and let the compiler check the RE correctness after the tokenization, generating the corresponding *Abstract-Syntax-Tree (AST)*, an optimizable high-level syntactic structure. Currently supported POSIX and PCRE operators are: characters alternation and concatenation; character classes ([abc]), ranges ([a-z]), their negation ([^abc]), and their short end (e.g., \w); the any character except \n (.); the bounded (? , {n}, {n,m}) and unbounded (*, +, {n,}) quantifiers with their lazy options (e.g., {n,}?); the escaping of characters with \.

Middle-End: Lowering and Optimizing the REs. The middle-end transforms the corresponding AST to an ISA-oriented representation. Then, it removes over-parenthesized sub-RE, assuming the ISA default AND operation among two consecutive instructions.

Although the ISA primitives contain at most four characters in the reference (limiting long string matching operations) the implicit AND among consecutive instruction lets us overcome this limitation. Specifically, it implies that two or more derived groups of AND operations behave as a bigger AND comprising all the characters. Similarly, we group OR and RANGE expressions by four characters as for the ISA limit. Differently from the AND case and supposing the RE exceeds this limits, the compiler chain them into a *complex* OR chain with subgroups of *base* OR with a maximum of four operands per node. This approach supports an indefinite number of characters in a single base expression, overcoming the ISA-limited reference. As the ISA offer a single quantifier primitive, the compiler transform Kleene operators (i.e., * and +) into the equivalent unbounded representation (i.e., {1,} and {0,}). Likewise, to reduce the primitives required, the compiler translate other ISA-unsupported primitives (e.g., \d or .) into a sequence of supported ones. For instance, the \w is translated in its equivalent representation [a-zA-Z0-9_], or the . translates into [^\n].

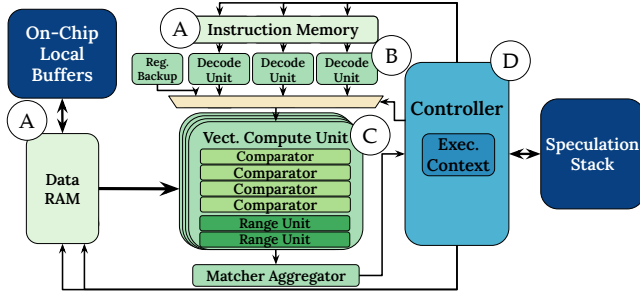


Figure 3: Single-core μ -architecture: (A) the *memories* for instructions and data, (B) the *decode units* prefetch the instructions for any change in the control flow, (C) the *vectorized compute unit* and a *matcher aggregator* for high-performance simple operators, (D) the *controller* and the *speculation stack* deal with complex operators, handle the matching procedure, and feature speculation approach to alternative paths.

These transformations output an *Intermediate Representation* (IR) which defines a compact ISA-like version of the REs. After the latest optimized syntax tree step, the compiler decorates the IR with relative jumps calculated on the exact instructions number. The IR traversal strategy follows the depth-first strategy, assigning the address to the leaves of a node before moving to the adjacent one.

Back-End: Operation Fusion and Binary Code. Finally, the back-end translates the resulting IR into the executable binary and, whenever possible, applies further architectural-aware optimizations allowed by the ISA, such as merging a closing sub-RE operator (e.g., `)`) with a base one (e.g., `AND`), creating a single instruction. This fusion is possible because base operators leverage the reference to encode the characters to scan, while the complex closing operators do not use this field §4. However, when two consequent closing operators are used in the RE, only the one nearest to the base operator is merged with it. In contrast, the outermost closing one requires additional instructions. The final output is a binary loadable into ALVEARE instruction memory and then executed.

6 ARCHITECTURAL ORGANIZATION

The RE-tailored DSA is optimized on top of the proposed ISA (§4). It executes a compiled RE (§5) and analyzes a data stream, producing the matching results. Figure 3 illustrates the architectural organization and its fundamental components.

Feeding the RE Matching Process via Memories (A). The architecture embeds two distinct memories for instructions and data to analyze (Figure 3 (A)). Specifically, the instruction memory stores the compiled binary code. This memory receives three addresses simultaneously and handles all the possible instruction flows in a RE: *sequential*, *backward*, and *forward* jumps. Thus, the DSA prefetches these instructions for the next cycle and decides which one to execute. In this way, each instruction completes at each cycle, following the RISC-based ISA principle.

Conversely, the data memory stores the data to analyze in a 2-level memory hierarchy to handle the access pattern. The on-chip local buffer stores the entire data chunk to analyze and continuously feeds the data to a smaller RAM until the data stream to explore is

consumed. Specifically, the smaller RAM interacts directly with the execution core and handles two addresses. The first address targets the latest sub-match address. We exploit it whenever a mismatch occurs, and the execution has to restart from the last match of the small RAM. Instead, the second address refers to the current pointer in the data stream. For this address, the data memory outputs the necessary number of chars to execute a base operator in the next cycle, compatibly with the number of vectorial compute units.

Preparing the Instructions: Decode Unit (B). Given that RE matching is highly data dependent and can choose an execution flow based on the sub-matching outcome, ALVEARE has three decode units that process the three instructions coming from the instruction memory at each cycle (Figure 3 (B)). Given the three prefetched instructions these units transform the 43 bits instruction into three different outputs: the reference, a signal to indicate which of the data in the reference are valid (if a base operation), and the one-hot decoded opcode. Then, the controller selects which prefetched instruction will pass to the following stage according to the matching outcome and on the operation. Additionally, we save the first instruction decoded in a backup register that is essential in case of a complete sub-matching failure to restart the RE procedure.

High-Performance Matching: Execution (C). Once the decoded instruction and the data stream are ready for analysis, the execution stage begins. This stage, building on top of a *vectorial unit* and an *aggregator unit* (Figure 3 (C)), processes the base ISA primitives (§4). On the one hand, the vectorial unit embeds four compute units (CUs), each of which includes four comparators that parallelly process the AND, the OR, and two RANGE units. Each additional CU implies an additional character analyzed in an overlapping fashion such that we find all the possible matches in the data stream. For instance, having four CUs means a maximum of analyzed data of seven characters (i.e., 4 chars for each CU with an overlapping of 3 chars with the adjacent: $\#Comparators + 1 \cdot (\#Compute_Units - 1)$).

On the other hand, the aggregator unit has a local view of the execution and coordinates the CUs. Specifically, it reorders the single operator results for each CU and raises a match in the following cases: if at least one of the comparators has a match for the OR; if one of the two range units has a match for the RANGE; if all the comparators of the CU have a match for the AND. Considering that the NOT operator is composable with the OR and RANGE, this unit inverts the overall results if needed.

ALVEARE Core: Controller and Speculation (D). The controller is at the core of the microarchitecture (Figure 3 (D)). It is a centralized control unit responsible for executing the complex operators of the ISA and managing the overall RE execution, which is highly data-dependent and embarrassingly sequential. To guarantee a correct execution flow, the controller must select the data to analyze and the instruction to execute among the prefetched ones. The expected flow for REs retrieves the data and executes the instructions sequentially. However, the complex operators, such as quantifiers or sub-REs alternation, change the data and instructions flows depending on sub-match occurrences. Moreover, to fully support the quantifiers modalities employed by the widespread PCRE standard, we designed the controller with a speculative approach with two modalities (i.e., greedy and lazy). The speculation allows the flow alteration, forcing the repetitions of the sub-pattern or

Table 2: ALVEARE ISA advanced primitives improvements.

Microbench. RE	Minimal Primitives	Advanced Primitives	Code/Clock Cycles Reduction
[a-zA-Z]	26	1	26×
[DBEZX]{7}	28	6	4.66×
.{3,6}	1160	2	580×
[^]*	66	2	33×

moving on to the next one depending on the modality and repetition requirements. Indeed, once entered into a sub-RE, the controller repeats the pattern as much as possible in the greedy mode (it speculates on the re-match of the sub-RE) or the least in the lazy one (it speculates on the match after the sub-RE), always satisfying the minimum number of repetitions.

To perform speculation, the controller exploits an additional stack memory structure to keep track of the alternative paths in the execution flow if a mismatch occurs during the speculation. Whenever a complex opening operator is encountered, the controller pushes the execution status information to the stack. In this way, we save a snapshot of the essential information to recover from the speculation in a rollback situation. The snapshot includes the quantification bounds (minimum and maximum), the current matches number of a quantifier, the possible sub-matching state of the architecture, the latest matched character position, and the current address in the data stream (i.e., before entering in the sub-pattern).

Scaling Out to a Multi-Core. We design a scale-out version replicating independent ALVEARE cores with private local memories. Such multi-core architecture operates on the same RE and searches for that pattern on different data stream portions. In this way, we can parallelize at the stream of data level and search for the pattern more effectively through a divide-and-conquer approach. The constraint for such scale-out is the area devoted to the multi-core (§7.2). Indeed, the data and instruction (i.e., the REs) memories are decoupled into two distinct private memories.

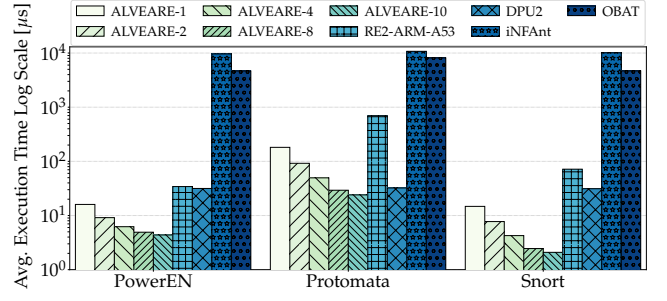
7 EVALUATION

We evaluate the benefits of adopting the advanced ISA primitives (§7.1) and ALVEARE against state-of-the-art solutions in embedded, near-data, and offloading scenarios (§7.2).

7.1 ALVEARE ISA Reduces Code and Cycles

We select representative RE microbenchmarks from [23] focusing on operators beyond the minimal set of regular language and widely employed by the standards. We compare these REs compiled with their minimal representation (i.e., performing compiler-based unfolding) and with our advanced ISA primitives. Table 2 shows the reduction of code size (excluding the EoR) and, being RISC-based, the cycles required to execute the whole set of instructions.

Although in the first RE our ISA shrinks `a|...|z|A|...|Z` alternations into 26 primitives thanks to the vectorization, it also offers the RANGE to express them into a single operation compactly. The second RE requires the concatenation of seven explicit `(D|B|E|Z|X)` repetitions, while our ISA compresses the quantification via the homonymous primitive with a 4.66× reduction. The third RE raises

**Figure 4: Execution time, the lower, the better (log scale). Top speedup of 7× (A53), 15× (DPU), 356× (GPU).**

the complexity since the `.` advanced operation translates in all the ASCII (128 chars in this case) but the “\n” repeated between 3 and 6 times. Nonetheless, we can reduce its cost by expressing it as `[^n]` along the quantification primitive, with a 580× gain. Finally, the last RE expresses as before all the characters (but the space one) repeated zero or more times, which we represent with the GREEDY QUANTIFICATION of a NOT RANGE, for a 33× reduction.

7.2 Setup and Literature Comparison

We implement our DSA in VHDL on an *embedded* Ultra96v2 board (AMD Zynq XCZU3EGA484), generate the bitstream with Vivado 2019.2, run it at 300 MHz, and exploit PYNQ 2.7 framework [1] for Host-DSA communications. We scale out from one to ten cores, as it is the maximum number fitting our FPGA resources due to the linear BRAM scaling (6.71% to 67.13%) and the sublinear LUT scaling (11.39% to 84.65%), and measure their impact.

State of the Art solutions. We focus the analysis on scenarios where CPUs have limited computing and power capabilities, e.g., embedded systems, or processing near to data sources is paramount to reduce the data transfer overhead, e.g., network-related, requiring data offloading is mandatory to alleviate data-intensive burdens. Within this context, we select Google RE2 [10], which is a cross-platform memory-safe approach to REs, and run it on the Ultra96 ARM A53 with -O3 optimizations, as embedded CPU solution. Then, we select the NVIDIA Bluefield 2 DPU [2], a commercially available system comprising an accelerator for REs as near-data solution. Finally, to represent offloading scenarios, we run on an NVIDIA V100 the first GPU-based algorithm, *iNFAnt*, and the State of the Art on GPUs, *OBAT* with *hotstart* optimization [12].

Benchmarks. We evaluate three representative benchmarks of ANMLZoo [23]. PowerEn, a synthetic benchmark from IBM for the evaluation of the Power Edge of Network SoC [22]. Protomata for research of protein patterns and one of the most complex benchmark in the suite [18]. Finally, Snort [3] a production-ready Deep Packet Inspection from CISCO. We chose the 1MB dataset and 200 REs randomly selected after excluding bad-formed REs. For fairness, we consider the DPU memory limits of 16KB input chunks.

Key Performance Indicators (KPI). We consider for each benchmark the average execution time and energy efficiency. For each RE executed, we assess the matching time after memories loading and repeat the experiments ten times to improve the stability. Considering the power, we use the V100 thermal design

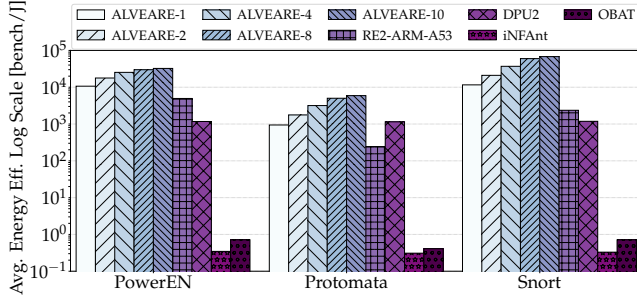


Figure 5: Energy efficiency, the higher, the better (log scale). Top gains: 29× (A53), 57× (DPU), orders of magnitude (GPU).

power, given lack of physical access, and for the A53, the DPU, and ALVEARE, we instrument via a Voltcraft 4000 measuring chip, on-board DRAM, and other parts of the system. The whole Ultra96 board with a 10-core ALVEARE consumes 7.05W, against the 27W of the DPU board and the 5.9W for the A53. We calculate the energy efficiency per benchmark as $Energy_Eff_{avg} = \frac{1}{Exe_Time_{avg} \cdot Power_{avg}}$.

Execution Time. Figure 4 shows that the 10-core ALVEARE achieves the best improvements against the single-core with a speedup of 3× for the synthetic PowerEN, while on real-life benchmarks (i.e., Protomata and Snort), around 7×. Nevertheless, the single-core is more than enough to perform better than RE2 in all the benchmarks with speedups from 2.13× to 4.86×, while the 10-core from 7.83× to 34.68×. Instead, the DPU features a divide-and-conquer approach via multi-threaded hardware to improve the performance. However, ALVEARE 10-core outperforms it in all the benchmarks with a peak speedup of 15.11× for real-world Snort. Conversely, GPU-based algorithms exhibit high processing times, at least two orders of magnitude higher than the others, hindering their usage in real scenarios. Indeed, our 10-core achieves a minimum speedup of 356× over OBAT on Protomata.

Energy Efficiency. Figure 5 illustrates the resulting efficiency of the whole systems. ALVEARE single core can, at worst, be in line with the State of the Art or improve the computation efficiency of a 9.78× on Snort. The top-performing ALVEARE 10-core architecture always delivers energy efficiency improvements, with peaks of 57.88× and 29× against the DPU and the A53. Finally, the single-core overwhelms GPU solutions by four order of magnitude.

Overall, ALVEARE demonstrates remarkable performance and energy efficiency improvements that make the proposed solution highly attractive for constrained and near-data scenarios, where offloading tasks such as RE matching is paramount.

8 CONCLUSION

This paper presents a comprehensive HW/SW framework called ALVEARE. We devise a novel ISA to support advanced RE primitives widely adopted by POSIX and PCRE standards while combining a compilation flow along a domain-specialized microarchitecture to simplify their design and balancing the efficiency-flexibility trade-off. Our evaluation demonstrates top speedup of 34× and efficiency boost of 57× against RE2-A53 and DPU (§7.2), representing a viable option in embedded or near-data scenarios.

ACKNOWLEDGMENTS

We thank the anonymous reviewers, L. Cicolini, E. D’Arnese, E. Del Sozzo, G. Sorrentino, and G. Antichi for precious feedback, and AMD University program, NVIDIA Academic HW Grant Program, Oracle Cloud Infrastructure and Oracle for Research for support.

REFERENCES

- [1] AMD. 2021. PYNQ v2.7. https://github.com/Xilinx/PYNQ/tree/image_v2.7.
- [2] Idan Burstein. 2021. Nvidia Data Center Processing Unit (DPU) Architecture. In *2021 IEEE Hot Chips 33 Symposium (HCS)*. 1–20.
- [3] Cisco. 2023. Snort Intrusion Prevention System (IPS). <https://www.snort.org/>.
- [4] Davide Conficconi, Emanuele Del Sozzo, Filippo Carloni, Alessandro Comodi, Alberto Scolari, and Marco Domenico Santambrogio. 2023. An Energy-Efficient Domain-Specific Architecture for Regular Expressions. *IEEE Transactions on Emerging Topics in Computing* (2023).
- [5] Paul Dlugosch, Dave Brown, Paul Glendenning, Michael Leventhal, and Harold Noyes. 2014. An efficient and scalable semiconductor architecture for parallel automata processing. *IEEE Transactions on Parallel and Distributed Systems* (2014).
- [6] Yuanwei Fang, Tung T Hoang, Michela Becchi, and Andrew A Chien. 2015. Fast support for unstructured data processing: the unified automata processor. In *Proceedings of the 48th International Symposium on Microarchitecture*.
- [7] Domenico Ficaro, Andrea Di Pietro, Stefano Giordano, Gregorio Procissi, Fabio Vitucci, and Gianni Antichi. 2010. Differential encoding of DFAs for fast regular expression matching. *IEEE/ACM Transactions on Networking* (2010).
- [8] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, et al. 2018. Azure Accelerated Networking: SmartNICs in the Public Cloud. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*.
- [9] Vaibhav Gogte, Aasheesh Kolli, Michael J. Cafarella, Loris D’Antoni, and Thomas F. Wenisch. 2016. HARE: Hardware accelerator for regular expressions. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
- [10] Google. 2020. Google re2. <https://github.com/google/re2>.
- [11] Lingkun Kong, Qixuan Yu, Agnishom Chattopadhyay, Alexis Le Gaunec, Yi Huang, Konstantinos Mamouras, and Kaiyuan Yang. 2022. Software-hardware codesign for efficient in-memory regular pattern matching. In *ACM PLDI*.
- [12] Hongyuan Liu, Sreepathi Pai, and Adwait Jog. 2020. Why gpus are slow at executing NFAs and how to make them faster. In *ACM ASPLOS*.
- [13] Marziyeh Nourian, Xiang Wang, Xiaodong Yu, Wu Feng, and Michela Becchi. 2017. Demystifying automata processing: GPUs, FPGAs or Micron’s AP? In *Proceedings of the International Conference on Supercomputing, ACM*.
- [14] Daniele Parravicini, Davide Conficconi, Emanuele Del Sozzo, et al. 2021. CICERO: A Domain-Specific Architecture for Efficient Regular Expression Matching. *ACM Transactions on Embedded Computing Systems (TECS)* (2021).
- [15] Junqiao Qiu, Lin Jiang, and Zhijia Zhao. 2020. Challenging sequential bitstream processing via principled bitwise speculation. In *ACM ASPLOS*.
- [16] Junqiao Qiu, Xiaofan Sun, Amir Hossein Nodehi Sabet, and Zhijia Zhao. 2021. Scalable FSM parallelization via path fusion and higher-order speculation. In *ACM ASPLOS*.
- [17] Reza Rahimi, Elaheh Sadredini, Mircea Stan, and Kevin Skadron. 2020. Grapefruit: An Open-Source, Full-Stack, and Customizable Automata Processing on FPGAs. In *International Symposium on Field-Programmable Custom Computing Machines*.
- [18] Indranil Roy and Srinivas Aluru. 2014. Finding motifs in biological sequences using the micron automata processor. In *IEEE IPDPS*.
- [19] Elaheh Sadredini, Reza Rahimi, Marzieh Lenjani, Mircea Stan, and Kevin Skadron. 2020. Impala: Algorithm/architecture co-design for in-memory multi-stride pattern matching. In *International symposium on HPCA*.
- [20] Arun Subramanian, Jingcheng Wang, Ezhil RM Balasubramanian, David Blaauw, Dennis Sylvester, and Reetuparna Das. 2017. Cache automaton. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*.
- [21] Lenka Turoňová, Lukáš Holík, Ondřej Lengál, Olli Saarikivi, Margus Veanes, and Tomáš Vojnar. 2020. Regex matching with counting-set automata. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020).
- [22] Jan Van Lunteren, Christoph Hagleitner, Timothy Heil, Giora Biran, Uzi Shvach, and Kubilay Atas. 2012. Designing a programmable wire-speed regular-expression matching accelerator. In *International Symposium Microarchitecture*.
- [23] Jack Wadden, Vinh Dang, Nathan Brunelle, Tommy Tracy II, et al. 2016. ANMLzoo: a benchmark suite for exploring bottlenecks in automata processing engines and architectures. In *IEEE International Symposium on Workload Characterization*.
- [24] Xiang Wang, Yang Hong, Harry Chang, Kyoungsoo Park, Geoff Langdale, Jiayu Hu, and Heqing Zhu. 2019. Hyperscan: a fast multi-pattern regex matcher for modern cpus. In *USENIX Symposium on NSDI*.
- [25] Jackson Woodruff and Michael FP O’Boyle. 2021. New Regular Expressions on Old Accelerators. In *58th ACM/IEEE Design Automation Conference (DAC)*.
- [26] Chengcheng Xu, Shuhui Chen, Jinshu Su, et al. 2016. A survey on regular expression matching for deep packet inspection: Applications, algorithms, and hardware platforms. *IEEE Communications Surveys & Tutorials* (2016).

Addition is Most You Need: Efficient Floating-Point SRAM Compute-in-Memory by Harnessing Mantissa Addition

Weidong Cao^{1,*}, Jian Gao^{2,*}, Xin Xin³, Xuan Zhang²

¹The George Washington University; ²Northeastern University; ³University of Central Florida

ABSTRACT

The compute-in-memory (CIM) paradigm holds great promise to efficiently accelerate machine learning workloads. Among memory devices, static random-access memory (SRAM) stands out as a practical choice for its exceptional reliability in the digital domain and excellent scalability. Recently, there has been a growing interest in accelerating floating-point (FP) deep neural networks (DNNs) with SRAM CIM due to their critical importance in DNN training and high-accurate inference. This paper proposes an energy-efficient SRAM CIM macro for FP DNNs. To achieve the design, we identify a lightweight approach that decomposes conventional FP mantissa multiplication into two parts: mantissa sub-addition (sub-ADD) and mantissa sub-multiplication (sub-MUL). Our study shows that while mantissa sub-MUL is compute-intensive, it only contributes to the minority of FP products, whereas mantissa sub-ADD, although compute-light, accounts for the majority of FP products. Recognizing “Addition is Most You Need”, we develop a novel hybrid-domain SRAM CIM macro to accurately handle mantissa sub-ADD in the digital domain while improving the energy efficiency of mantissa sub-MUL using analog computing. Experiments with the MLPerf benchmark show its remarkable improvement in energy efficiency on average by $3\times \sim 3.6\times$ ($2.5\times \sim 3.1\times$) in inference (training) compared to a fully digital baseline without any accuracy loss, showcasing its great potential for FP DNN acceleration.

1 INTRODUCTION

The compute-in-memory (CIM) has emerged as a highly promising computing paradigm by colocating computation and storage in close proximity [1, 2, 4, 10, 16, 21]. In particular, it has shown remarkable energy efficiency in accelerating a variety of machine learning tasks, ranging from image classification and object recognition to natural language processing [14, 16, 19, 21]. Among the various memory devices explored [6, 8, 14, 16, 17, 19, 21, 22], static random-access memory (SRAM) stands out in building up CIM systems [14–16, 19, 21] for practical applications due to its exceptional reliability in the digital domain while maintaining superior performance, power, and area. Although both academia [14, 16] and industry [3, 5] have been advancing the performance of SRAM CIM, most endeavors [3, 5, 14] have been limited to accelerate quantized deep neural network (DNN) models for inference [20] due to their relatively simple arithmetic operations, lightweight computing unit, and, accordingly, high energy efficiency compared to their floating-point (FP) counterparts [12].

*Equal contribution.

Correspondence: weidong.cao@gwu.edu & xuan.zhang@northeastern.edu.

This work is partially supported by NSF CCF #1942900.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Recently, there has been a growing interest in studying FP SRAM CIM macros [16, 19, 21]. This is because DNN models have been predominantly based on FP arithmetic operations to achieve the highest training quality [11, 12]. In addition, for mission-critical applications such as autonomous driving, security robots, and defense drones, quantized DNN models do not always ensure the stringent accuracy requirement [6, 12, 13]. As a result, efficient CIM systems to accelerate FP DNN models are highly coveted and crucial for unlocking the full power of machine learning (ML). Unfortunately, only a few SRAM CIM macros have been proposed to speed up FP DNN models for edge or cloud applications [16, 19, 21]. Despite the notable performance improvements compared to GPUs and systolic neural processing units, there is still substantial untapped potential to further optimize the energy efficiency of FP CIM macros. In particular, current SRAM CIM macros [16, 19, 21] that focus on boosting vector-matrix multiplication (VMM) often treat FP multiplication and FP addition as the basic units of computation in the digital domain. They implement these FP operations by overminimizing computation error on every single bit operation. Our studies show that fully accurate computations incur energy inefficiency and are not necessary for FP DNNs due to their inherent resilience to small perturbations. Such insights provide us with promising opportunities to enhance the performance of FP SRAM CIM macros.

This paper proposes an efficient SRAM CIM macro to accelerate FP DNNs by harnessing the inherent advantage of FP arithmetic. By dividing the mantissa multiplication into two parts, i.e., mantissa sub-addition (sub-ADD) and sub-multiplication (sub-MUL), we find that sub-ADD contributes to $\sim 75\%$ of FP products while consuming $<10\%$ of the total energy (Section. 3.1). This insight of “Addition is Most You Need” motivates our design approach for the proposed FP SRAM CIM macro: dedicating digital resources to guarantee the accuracy of lightweight mantissa sub-ADD, while exploiting energy-efficient analog computing to reduce the complexity of mantissa sub-MUL. This hybrid-domain acceleration achieves state-of-the-art energy efficiency without compromising the accuracy of DNN models, outperforming the traditional fully-accurate digital-only baseline [21]. Key contributions of this work are listed below.

- We identify a lightweight mechanism to accelerate FP DNNs by decomposing FP mantissa multiplication into two parts: accuracy-oriented mantissa sub-ADD and efficiency-oriented mantissa sub-MUL.
- We tailor a hybrid-domain SRAM CIM macro to implement the proposed mechanism by placing mantissa sub-ADD in the digital domain and performing mantissa sub-MUL in the analog domain.
- Detailed circuit- and microarchitecture-level features of the proposed FP SRAM CIM macro, such as local computing cells and computation flow, are elaborated.
- Experimental evaluations reveal that the proposed hybrid-domain SRAM CIM macro can improve energy efficiency by $3.0\times \sim 3.6\times$ ($2.5\times \sim 3.1\times$) in inference (training) without degrading accuracy compared to the FP digital baseline.

2 BACKGROUND

2.1 Floating-Point Arithmetic Operations

We first briefly introduce the essential FP arithmetic operations underpinning FP DNN models.

FP format: A general FP number in scientific notation is expressed as $f = (-1)^S \cdot 2^E \cdot 1.M$. Here, S ($S = 0$ or $S = 1$), E , and M ($M \in (0, 1)$) represent the sign, exponent, and mantissa (fraction) of the number, respectively. Note that ‘1’ before ‘ M ’ is a hidden bit, which is not explicitly shown in the binary format of an FP number and E is the actual exponent with an offset to the exponent encoded in the standard format. Taking the IEEE half-precision FP format (FP16) as an example, S is 1-bit, E is 5-bit, M is 10-bit (refer to Figure 2(d)).

FP multiplication: The multiplication of FP numbers is straightforward, i.e., exponent summation (①) and mantissa multiplication (②), as shown below using two positive numbers for simplicity.

$$(2^{E_0} \cdot 1.M_0) \cdot (2^{E_1} \cdot 1.M_1) = \underbrace{(2^{E_0+E_1})}_{\text{①}} \cdot \underbrace{(1.M_0 \cdot 1.M_1)}_{\text{②}}. \quad (1)$$

FP addition: However, the addition of FP numbers is non-trivial, which is expressed as

$$2^{E_0} \cdot 1.M_0 + 2^{E_1} \cdot 1.M_1 = 2^{E_{\max}} \cdot \underbrace{\left\{ \begin{array}{l} \underbrace{2^{E_0-E_{\max}} \cdot 1.M_0}_{\text{②}} \\ + \\ \underbrace{2^{E_1-E_{\max}} \cdot 1.M_1}_{\text{②}} \end{array} \right\}}_{\text{③}}. \quad (2)$$

It involves alignment first, i.e., finding the maximum exponent, $E_{\max} = \max\{E_0, E_1\}$, among all exponents (①) and obtaining the exponent difference, $E_{0(1)} - E_{\max}$, between each exponent and the maximum one (②). Subsequently, the mantissa part would be shifted towards the right according to the exponent difference and then summed together (③). The sum will then undergo extra processes like truncation to comply with the standard FP format.

FP vector-matrix multiplication: By generalizing Eq. (2) to the accumulation of n FP numbers, where each number, $2^{E_i} \cdot 1.M_i$ is assumed to be a product of a weight-activation pair, i.e., $(-1)^{W_{S,i}} \cdot 2^{W_{E,i}} \cdot 1.W_{M,i}$ and $(-1)^{X_{S,i}} \cdot 2^{X_{E,i}} \cdot 1.X_{M,i}$ based on Eq. (1) (i.e., $E_i = W_{E,i} + X_{E,i}$ and $1.M_i = 1.W_{M,i} \cdot 1.X_{M,i}$), the vector-matrix multiplication (VMM) of FP numbers is achieved, which is the backbone operation of FP DNN models. FP VMM can support the computation of DNN models with the highest accuracy and the best training quality [12]. Therefore, efficient FP VMM acceleration on hardware is highly desirable.

2.2 Compute-in-Memory for FP DNNs

Compute-in-Memory (CIM) to accelerate FP VMM is conceptually illustrated in Figure 1. First, the alignment is carried out by summing the exponent parts of weight-activation pairs ($W_{E,i}$ and $X_{E,i}$, Step ①) and obtaining the exponent difference between each exponent sum, E_i , and the maximum one, $E_{\max}$ (Step ②). Subsequently, exponent differences ($E_i - E_{\max}$) are used to shift the mantissa parts of activations, which are then used for mantissa multiplication and accumulation (Step ③). Note that these steps in Figure 1 exactly match those shown in Eq. (2).

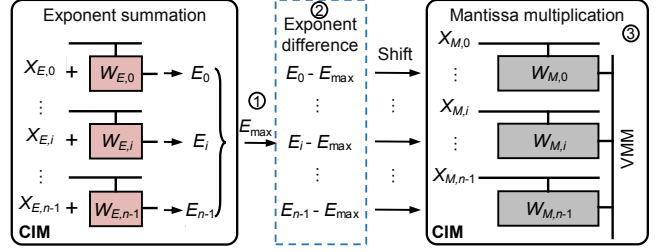


Figure 1: Illustration of CIM for FP VMM acceleration. $X_{E,i}$ and $W_{E,i}$ are the exponent parts of activation and weight. $X_{M,i}$ and $W_{M,i}$ are the mantissa parts of activation and weight. ①, ②, and ③ are circuit-level representations of the steps in Eq. (2).

Previous work has explored CIM acceleration for FP DNN models with emerging non-volatile memory (NVM) devices [6, 9]. Despite their great promise compared to GPUs and systolic neural processing units, such NVM devices are still in the early stages of development and are not reliable enough for practical applications. Recently, static random-access memory (SRAM) has emerged as the cornerstone in the design of CIM systems for practical applications because of its exceptional accuracy in digital computing, coupled with superior performance, power efficiency, and area [14, 16, 19, 21]. However, exploration of FP SRAM CIM macros has been limited to a few edge/cloud applications [16, 19, 21]. As an example, a prior work [16] proposes a reconfigurable FP/INT CIM processor to enable flexible support of BFloat16 (BF16)/FP32 and INT8/16 in the same digital CIM macro. Another work [21] divides FP operations into high-efficiency intensive-CIM and flexible sparse-digital parts, as it observes that most exponents of FP data are clustered in a small range. Circuit-level techniques, such as the time-domain exponent summation mechanism, are also used to improve the energy efficiency of CIM macros [19]. Despite these different implementations, they primarily focus on accurately accelerating almost every exponent summation and mantissa multiplication in the digital domain, resulting in remarkable, yet unnecessary energy waste.

Our analysis reveals that by leveraging the inherent advantage of FP arithmetic operations and exploiting the inherent resilience of DNN models to small computation errors, there is significant untapped potential to leverage approximating computing (e.g., analog computing) to accelerate FP DNNs without degrading their accuracy (Section 3.1). This revelation opens valuable opportunities to further enhance the performance of FP SRAM CIM macros.

3 APPROACH

3.1 Insight: Addition is Most You Need

By carefully examining FP arithmetics, we find a lightweight manner to allow hardware-friendly implementation of SRAM CIM macros with enhanced energy efficiency while maintaining the accuracy of FP DNNs. To show the key idea, we decomposed the conventional FP mantissa multiplication in Eq. (1) into two parts as

$$(1.M_0 \cdot 1.M_1) = (1 + M_0) \cdot (1 + M_1) = \underbrace{(1 + M_0 + M_1)}_{\text{sub-ADD}} \cdot \underbrace{M_0 \cdot M_1}_{\text{sub-MUL}}, \quad (3)$$

where $(1 + M_0 + M_1)$ and $(M_0 \cdot M_1)$ are defined as mantissa sub-addition (sub-ADD) and mantissa sub-multiplication (sub-MUL). Following the decomposition and given $M_{0(1)} \in (0, 1)$, we study

the significance of sub-MUL in mantissa multiplication, i.e.,

$$\frac{M_0 \cdot M_1}{(1 + M_0) \cdot (1 + M_1)} = \frac{1}{(1 + 1/M_0) \cdot (1 + 1/M_1)} \leq \frac{1}{4}. \quad (4)$$

Here, the ‘=’ holds true if and only if $M_0 = M_1 \rightarrow 1$. Eq. (4) shows that for a single weight-activation pair, if the computation accuracy of sub-ADD can be ensured, the *total computation error of mantissa multiplication (also for the FP product) does not exceed 1/4 compared to its ground truth even by aggressively removing the sub-MUL*. Figure 5 shows our experimental results (the red dots) by throwing away sub-MUL operations in FP multiplication of a single weight-activation pair with exemplary FP DNN models from our benchmark. The experiment verifies that the total computation error resulting from removing all sub-MUL operations is bounded by 1/4, and *the error is smaller than 1/4 in most cases*.

The analysis here shows that although mantissa sub-MUL operations are computationally intensive, they often impact a minority of FP products. In contrast, mantissa sub-ADD, although computationally lighter, often constitutes the majority of FP products. And addition is much more energy efficient compared to multiplication. For example, INT8 addition consumes about 10% energy of INT8 multiplication according to previous work [7]. This pivotal insight of “Addition is Most You Need” inspires our FP SRAM CIM macro design strategy: **allocating digital resources to ensure the precision of compute-light mantissa sub-ADD and employing energy-efficient analog computing for compute-intensive mantissa sub-MUL**.

3.2 Hybrid-Domain FP SRAM CIM Macro

Microarchitecture overview: Figure 2(a) illustrates the microarchitecture of the proposed SRAM CIM macro (targeting FP16 DNN models as an example). It consists of a time-domain CIM exponent summation array (ESA, to add the exponent parts of weight-activation pairs), time-domain MAX identifier (to find the maximum exponent sum), exponent difference extractor (EDE, to extract the exponent difference between each exponent sum and the maximum one), time-to-digital processing unit (to convert the exponent difference in the time domain into digits), EDE-based input alignment unit (to shift mantissa parts of activations), hybrid-domain CIM mantissa VMM array (HD-MVA, for FP VMM), analog-to-digital converters (ADCs), shift-and-add (S&A) module, local digital adder tree, partial-product management (PM) unit, and other auxiliary modules such as the IO module and pipeline and timing control unit. For exponent summation and alignment, we implement them in the time domain, since previous work [18, 19] has shown that the time-domain implementation is more energy efficient as compared to other digital implementation manners. For mantissa multiplication, we develop a hybrid-domain SRAM CIM macro to precisely manage mantissa sub-ADD in the digital realm while enhancing the energy efficiency of mantissa sub-MUL through analog computing. **Computation flow:** The computation process of the proposed SRAM CIM Macro is shown in Figure 2(b), which contains seven key steps. In Step ①, ESA computes the sum of the exponent parts of each weight-activation pair in the time domain, i.e., $E_i = W_{E,i} + X_{E,i}$. In Step ②, the maximum exponent sum, $E_{\max}$, among all sums, is found by the time-domain MAX identifier. In Step ③, all exponent sums ($E_i, i = 0, \dots, n - 1$) and $E_{\max}$ are sent to the EDE unit, where

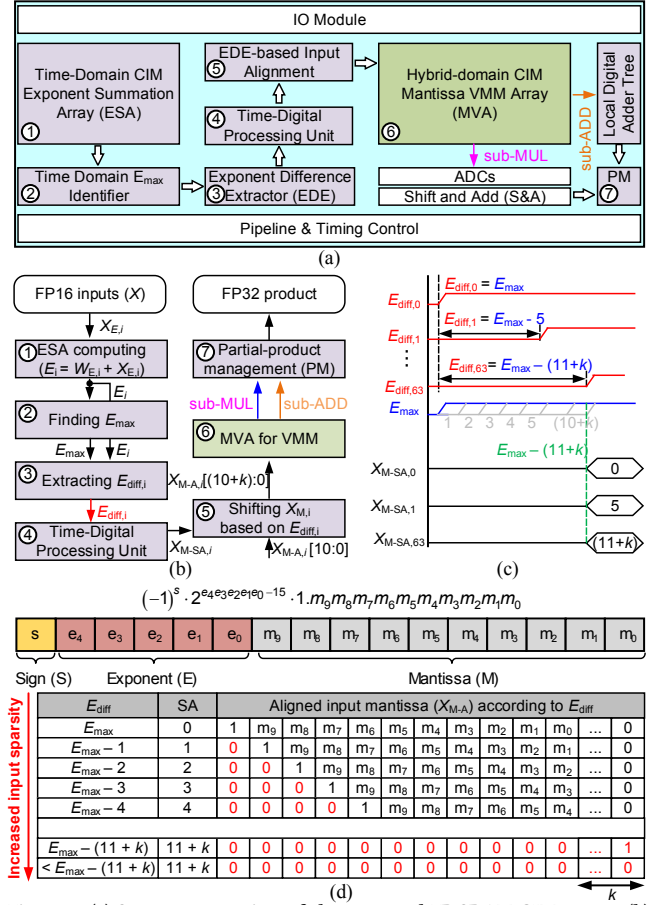


Figure 2: (a) Structure overview of the proposed FP SRAM CIM macro. (b) Computation flow. (c) Illustration of the exponent difference extraction in the time domain. (d) Illustration of mantissa shift based on the exponent difference (using FP16 as an example).

the exponent difference $E_{\text{diff},i}$ between E_i and $E_{\max}$ is obtained, i.e., $E_{\text{diff},i} = E_{\max} - E_i$. In particular, $E_{\text{diff},i}$ is calculated by measuring the time interval between the rising edges of E_i and $E_{\max}$ as shown in Figure 2(c). In Step ④, the time-to-digital processing unit converts $E_{\text{diff},i}$ from the time domain into a digital shift-amount (SA) value, $X_{M-SA,i}$, for each mantissa, $X_{M,i}$. The conversion relation between $E_{\text{diff},i}$ and $X_{M-SA,i}$ is straightforward as illustrated in Figure 2(c). With this $X_{M-SA,i}$, in step ⑤, the original mantissa $X_{M,i}$ is shifted to generate the aligned mantissa, $X_{M-A,i}$ with the same bit-width, i.e., $11 + k$. If $E_{\text{diff},i} > (11 + k)$, $X_{M-A,i}$ would become 0, allowing computation to be skipped. “11” is the bit width of the mantissa part in FP16 format (1 hidden bit plus 10 explicit mantissa bits). Note that k is a tuning value, allowing for a further trade-off between accuracy and energy efficiency [19], which is discussed in the next paragraph. Otherwise, if $E_{\text{diff},i} \leq (11 + k)$, $X_{M-A,i}$, as shown in Figure 2(d), the insert-0 behavior coupled with the growth of the exponent difference increases the bit-level sparsity, which reduces the energy consumption of the proposed HD-MVA with the input-sparsity-aware circuit. In step ⑥, MVA uses the aligned mantissa parts to achieve VMM with analog computing, sub-MUL, and digital computing, sub-ADD. Finally (Step ⑦), $E_{\max}$, the partial products from sub-MUL and sub-ADD are post-processed by the PM unit to obtain the final product in the standard FP32 format.

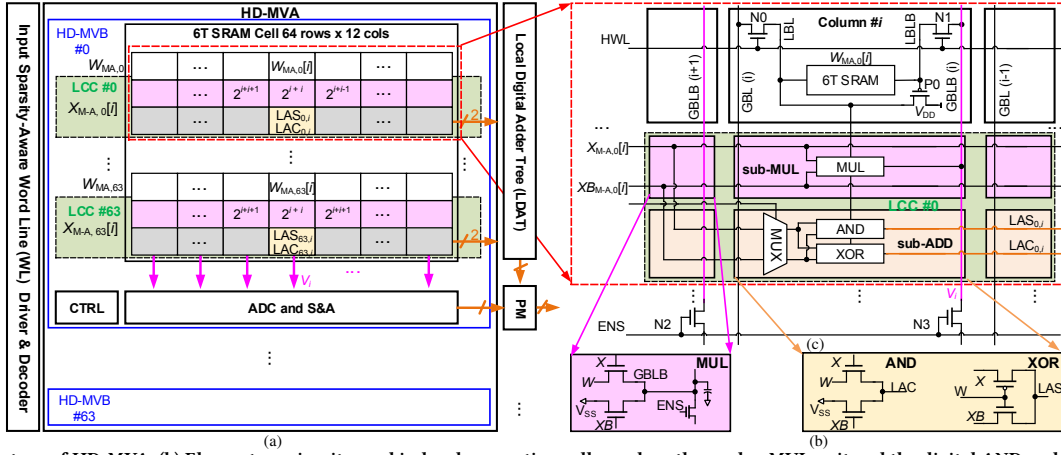


Figure 3: (a) Structure of HD-MVA. (b) Elementary circuits used in local computing cells, such as the analog MUL unit and the digital AND and OR gate. (c) Local computing cell to implement the proposed hybrid computing mechanism.

Ideally, the bit-width of an aligned activation mantissa is $(11 + E_{\max} - E_{\min})$, where $E_{\min}$ is the minimum exponent sum among all E_i . However, if $(E_{\max} - E_{\min})$ is too large, the product of a weight-activation pair whose exponent sum is in the neighbors of $E_{\min}$ could have a negligible contribution to the final product of the VMM. Therefore, we use the tunable k to find a proper shift-amount value for alignment, which can ensure the accuracy of the computation while further improving the energy efficiency. Additionally, we use a coarse pipeline to increase the throughput of the proposed CIM macro, where Steps ① to ④ are classified into pipeline stage one and the remaining steps are in pipeline stage two.

Sub-circuit design: The key circuit blocks to implement steps ① to ⑤ are similar to the previous work [18, 19], whose details are thus omitted here. Instead, we focus on introducing the proposed HD-MVA as shown in Figure 3(a). The HD-MVA contains 64 hybrid-domain CIM mantissa VMM banks (HD-MVB). Each HD-MVB is used for computing the product of aligned activation mantissa X_{M-A} and weight mantissa W_{MA} , i.e., $\sum (X_{M-A,i} \cdot W_{MA,i})$. Here, $X_{M-A,i}$ is $(11 + k)$ -b wide; $W_{MA,i}$ is 12-b wide and represented in 2's complement. To show the working mechanism of HD-MVB, we assume $k = 0$ for simplicity and take the circuit in Figure 3(b) as an example.

Each HD-MVB comprises a 6T-SRAM cell array (64 rows and 12 columns) and a local computing cell (LCC) associated with each row. In particular, each bit of $W_{MA,i}$ is stored in the same row but across different columns of the 6T-SRAM cell array in the order from the most significant bit (MSB) to the least significant bit (LSB). Each LCC includes a one-bit multiplication unit (MUL) for sub-MUL in the analog domain and a half-adder for sub-ADD in the digital domain. In each input cycle j , the row i sends a 2-b sum of $(X_{M-A,i}[j] + W_{MA,i}[j])$, i.e., $LAS_{i,j}$ and $LAC_{i,j}$, to the local digital adder tree (LDAT). Here, $LAS_{i,j}$ is the local-add sum bit, and $LAC_{i,j}$ is the local-add carry bit. In the same cycle, each column (global bitline bar, GBLB) also generates a partial product of $X_{M-A}[j]$ and W_{MA} . Across cycles, the LDAT accumulates partial sums of mantissa summation and the S&A logic accumulates the partial products of mantissa multiplication. Finally, PM combines these results to generate the standard FP32 product for subsequent processing.

Similarly to previous LCC structures [18, 19], the two pass-transistors ($N0/N1$) connect GLB/GBLB to the local bitline (LBL/LBLB) for read and write operations in SRAM mode. In standby mode, the horizontal wordline (HWL) is activated, i.e., $HWL = 1$, with GBLs pre-charging LBLs. In CIM mode, $N0/N1$ are switched off to decouple GBLs from LBLs. The selected WL is activated to read 1b

Table 1: Benchmark from the MLPerf Inference: Edge Benchmark Suite v3.1.

Task	Model	Dataset	Quality (99% FP32)
Image classification	ResNet50-v1.5	ImageNet	76.014% (top1 Acc)
Object detection	Retinanet	OpenImages	0.3755 mAP
Language processing	BERT-large	SQuAD v1.1	90.874% (f1_score)
Speech-to-text	RNNT	Librispeech	92.548% (1 - WER)

of weight mantissa (e.g., $W_{MA}[j]$), which is stored in the SRAM cell accessed with a large voltage swing. The multiplexer (MUX) is used to select a specific bit of X_{M-A} to be added to the corresponding bit of the weight mantissa in a cycle, whose control signal is from the decoder in the input module of the HD-MVA. The schematics of the MUL unit for sub-MUL operations and the AND/XOR gate for sub-ADD operations are illustrated in Figure 3(c). All of these circuits use a two-transistor structure to minimize area overhead.

4 EXPERIMENTS

4.1 Experimental Methodology

Benchmarks: We evaluate both the inference and training performance of the proposed FP SRAM CIM macro. Current FP SRAM CIM macros mainly target edge applications with FP16 DNN models [16, 19, 21]. We follow the convention and use the MLPerf Inference: Edge benchmark suite v3.1 [13] and MLPerf Training Benchmark Suite v3.1 [11] as our benchmarks. The inference benchmark includes image classification, object detection/segmentation, speech-to-text, language processing, and recommendation tasks, while the training benchmark extends them further to large language models and image generation. We select four inference tasks and one training task to showcase our proposed CIM macro's performance. Their associated datasets, models, and quality metrics for inference are listed in Table 1. The metrics for these models are top-1 accuracy for the image classification (ResNet50-v1.5), mean average precision (mAP) for the objection detection (Retinanet), f1_score which represents the harmonic mean of the precision and recall for the language processing (BERT-large), and 1-word error rate (WER) for the speech-to-text (RNNT).

Baseline: The fully digital architecture of the state-of-the-art FP SRAM CIM macro [21] is used as our baseline. This work implements FP operations in two parts: high-efficiency intensive-CIM and flexible sparse-digital parts, as it is observed that most of the exponents of FP data are clustered in a small range. This macro accelerates FP VMM in the digital domain without any approximate computation. For a fair comparison, we build the two architectures based on the same pipeline with 28nm CMOS technology

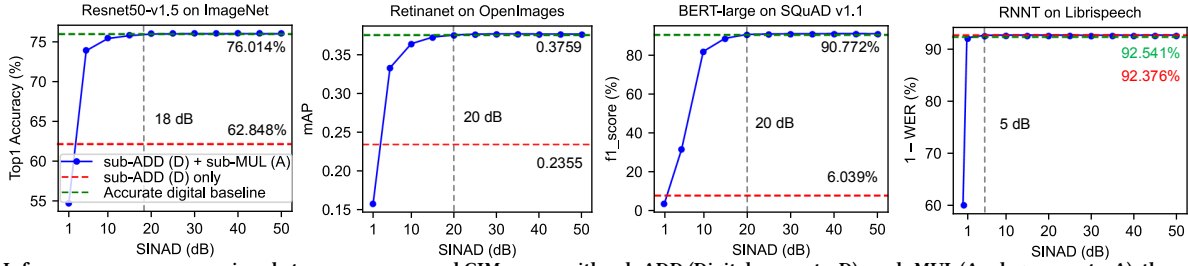


Figure 4: Inference accuracy comparison between our proposed CIM macro with sub-ADD (Digital compute, D) + sub-MUL (Analog compute, A), the macro with sub-ADD (Digital compute, D) only, and the accurate digital baseline with MLPerf Inference: Edge Benchmark Suite v3.1 Tasks.

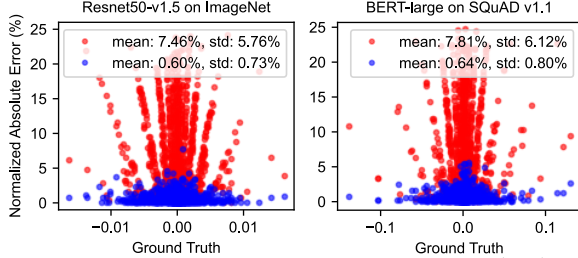


Figure 5: Noise/error characterization of our proposed macro (blue) and the macro with sub-ADD only (red, discarding sub-MUL) in FP element-wise multiplication with exemplary DNN layers from MLPerf. The ground truth is from the fully accurate digital baseline. Our macro achieves $\text{SINAD}_{\text{hw}} = 30$ dB with a much smaller computation error.

and use the same capacity of SRAM cells, i.e., 832kb. We use the same power supply $V_{\text{DD}} = 0.8\text{V}$ and the clock frequency of 200 MHz for them. We do not consider the tunable parameter k mentioned in Section 3.2 when comparing with two architectures, i.e., $k = (E_{\text{max}} - E_{\text{min}})$. We first evaluate the computation accuracy of the analog flow (for the sub-MUL operations) used in our proposed CIM macro. We then evaluate the accuracy/energy efficiency for the two designs. Finally, we present a summary.

4.2 Macro Accuracy Evaluation

Noise characterization of the analog dataflow: We first characterize the computation accuracy of the analog flow used for the sub-MUL operations. The proposed SRAM CIM macro employs an analog dataflow for sub-MUL operations and hence is subject to various hardware non-idealities, such as process, voltage, and temperature (PVT) variations, thermal noise, and quantization noise of ADCs. Together, these non-idealities act to influence the computation accuracy of the macro. We adopt the signal-to-noise and distortion ratio (SINAD) to represent the computation accuracy of the analog dataflow for sub-MUL operations [2]. To characterize the SINAD of our proposed CIM macro, we perform hundreds of such Monte Carlo (MC) simulations on hardware by using random weight-activation pairs, the errors statistics between the hardware computation results, D_{hw} , and the software ground truth, D_{sw} , can be obtained. The variation ϵ of the hardware noise is then expressed as $(\epsilon)^2 = P_{\text{noise}} = \text{mean}(D_{\text{hw}} - D_{\text{sw}})^2$. With the variations, the signal-to-noise and distortion ratio (SINAD) of the analog dataflow is characterized as $\text{SINAD}_{\text{hw}} = 10 \log_{10}((P_{\text{sig}}(D_{\text{sw}}) + P_{\text{noise}})/P_{\text{noise}})$. Intuitively, the smaller variation ϵ is, the higher the SINAD is, and the more accurate the computation is. The evaluations in Figure 5 are from thousands of simulations with exemplary DNN layers from MLPerf image classification and language processing tasks. The ground truth is the element-wise product from the accurate digital baseline. The statistics of the normalized absolute error between D_{hw} and D_{sw} of our proposed macro (blue) shows an equivalent SINAD_{hw} of 30 dB with analog computing. The results indicate that our proposed macro yields a mean normalized absolute error that is

around $12\times$ smaller compared to the one (red) of the sub-ADD-only computation (i.e., discarding sub-MUL).

Inference accuracy: To examine whether the SINAD_{hw} derived from our analog dataflow can support adequate system-level inference accuracy, we analyze the inference accuracy of an FP DNN model on the proposed CIM macro by sweeping the level of SINAD. In this way, we can obtain $\text{SINAD}_{\text{min}}$, the minimum SINAD required to achieve the software-equivalent inference accuracy. In particular, we apply a reverse method to inject equivalent software-level noise into each layer’s input activations of an FP DNN model, i.e., adding noise to D_{sw} with the corresponding SINAD.

Figure 4 demonstrates the effects of varying the SINAD on the inference accuracy of different DNNs with software-level sweeping, where the blue line represents the sweeping curve of our proposed CIM macro (sub-ADD (Digital compute, D) + sub-MUL (Analog compute, A)). Typically, a 20 dB SINAD is sufficient for all DNN models to reach the ideal inference accuracy. Especially, RNNT on Librispeech only needs 5 dB to achieve the accuracy of the digital baseline. We believe that the robustness of RNNT against the noise from the sub-MUL analog computing is attributed to its network architecture by using Long short-term memory (LSTM) cells. LSTMs have a unique architecture with input, output, and forget gates. These gates effectively regulate the flow of information, allowing the network to retain important information over long sequences and forget irrelevant data. This gating mechanism allows the errors to propagate through the network in a more controlled manner. Since our macro provides a SINAD as high as 30 dB, all inference accuracy can be guaranteed without degeneration. However, if the sub-MUL operations are excluded from the FP VMM, the accuracy of models significantly drops. As an example, the Top-1 accuracy of ResNet50-v1.5 on ImageNet decreases to 62.848% without the sub-MUL operations. The BERT-large on SQuAD v1.1 can even fail to work by throwing away the sub-MUL operations.

Training accuracy: We adopt the same manner as above to evaluate the training accuracy of an FP DNN model on the proposed CIM macro. We inject the noise based on the obtained SINAD_{hw} into the FP DNN models during the training to emulate the non-ideal hardware. Figure 6(a) shows a training example by using ResNet-50 v1.5 on ImageNet. The top-1 training accuracy with the proposed CIM macro is 75.91% with no accuracy loss to the digital baseline.

4.3 Macro System-Level Evaluation

Energy efficiency: We further evaluate the energy efficiency of our proposed CIM macro on FP DNN acceleration by comparing it to the fully digital baseline [21]. On average, our proposed CIM macro can improve energy efficiency by $3\times\sim 3.6\times$ ($2.5\times\sim 3.1\times$) for inference

If $\text{SINAD}_{\text{hw}} \geq \text{SINAD}_{\text{min}}$, we consider that the analog dataflow has sufficient computation accuracy to guarantee the system-level inference accuracy.

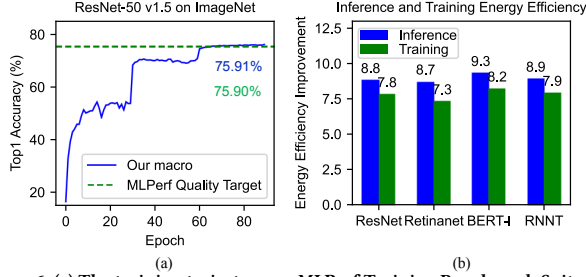


Figure 6: (a) The training trajectory on MLPerf Training Benchmark Suite v3.1 Image classification task with our proposed macro. (b) Normalized inference (training) energy efficiency to the digital baseline using model layers with a more dense distribution of exponents of FP weights/activations.

Table 2: Summary of performance of the proposed CIM macro for layers with dense exponent distribution or on average.

	Inference		Training	
	Accuracy	Energy Efficiency	ACC	EE
Baseline [21]	No loss	1×	No loss	1×
Our work (Dense)	No loss	8.7×~9.3×	No loss	7.3×~8.2×
Our work (AVG)	No loss	3×~3.6×	No loss	2.5×~3.1×

(training) compared to the baseline across various benchmark models. The improvement is mainly due to the efficient acceleration of sub-MUL operations in the analog domain. In particular, as shown in Figure 6, if the exponent distribution of FP weight/activations in a particular model layer is more dense, our macro is able to improve the energy efficiency by $8.7\times\sim 9.3\times$ ($7.3\times\sim 8.2\times$) during inference (training) compared to the baseline. This further improvement is primarily due to the lower cost of the mantissa alignment.

Performance and area: Since our proposed macro aligns the mantissa in the time domain, it can reduce the time of each pipeline stage, leading to a $1.23\times$ improvement in performance compared to the baseline. The local digital adder tree (LDAT) in our proposed macro is only used for the accumulation of partial sums of sub-ADD, which minimizes the area overhead. In contrast, the baseline requires area-intensive digital adder trees for the accumulation of all partial sums, and RISC-V CPU for the computation of weights and activations that are sparse in their distributions. Compared to this baseline, our proposed CIM macro achieves $1.43\times$ area efficiency.

Summary: Finally, we make a summary of the performance of the proposed CIM macro for FP DNN acceleration as shown in Table 2. It achieves better performance in terms of energy efficiency, latency, and area efficiency compared to baseline [21] with the same amount of computing resources. This comparison is preliminary and based on circuit simulations without detailed optimizations. Future efforts will focus on optimizing the hardware configuration of the proposed macro and evaluating it with silicon prototypes.

5 CONCLUSION

This paper has introduced an efficient SRAM CIM macro for FP DNNs. It splits FP mantissa multiplication into mantissa sub-addition (sub-ADD) and sub-multiplication (sub-MUL). It shows that sub-MUL is compute-intensive but contributes less to FP vector-matrix-multiplication (VMM), sub-ADD contributes more to FP VMM yet less compute-heavy. Leveraging this insight, we develop a hybrid-domain SRAM CIM macro that efficiently processes sub-ADD digitally and enhances sub-MUL’s energy efficiency through analog computing. Experiments show significant improvements in energy efficiency over digital baselines without hurting accuracy.

REFERENCES

- [1] Weidong Cao and Xuan Zhang. 2023. A/D Alleviator: Reducing Analog-to-Digital Conversions in Compute-In-Memory with Augmented Analog Accumulation. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5.
- [2] Weidong Cao, Yilong Zhao, and et al. 2022. Neural-PIM: Efficient Processing-In-Memory With Neural Approximation of Peripherals. *IEEE Trans. Comput.* 71, 9 (2022), 2142–2155.
- [3] Yu-Der Chih, Po-Hao Lee, Hidehiro Fujiwara, Yi-Chun Shih, Chia-Fu Lee, Rawan Naous, Yu-Lin Chen, and et al. 2021. 16.4 An 89TOPS/W and 16.3TOPS/mm² All-Digital SRAM-Based Full-Precision Compute-In Memory Macro in 22nm for Machine-Learning Edge Applications. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. 252–254.
- [4] William J. Dally, Yatish Turakhia, and Song Han. 2020. Domain-Specific Hardware Accelerators. *Commun. ACM* 63, 7 (jun 2020), 48–57.
- [5] Hidehiro Fujiwara, Haruki Mori, Wei-Chang Zhao, Mei-Chen Chuang, Rawan Naous, Chao-Kai Chuang, and et al. 2022. A 5-nm 254-TOPS/W 221-TOPS/mm² Fully-Digital Computing-in-Memory Macro Supporting Wide-Range Dynamic-Voltage-Frequency Scaling and Simultaneous MAC and Write Operations. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. 1–3.
- [6] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. 2019. FloatPIM: In-Memory Acceleration of Deep Neural Network Training with High Precision. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. 802–815.
- [7] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, and et al. 2021. Ten Lessons From Three Generations Shaped Google’s TPUv4i: Industrial Product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 1–14.
- [8] Heesu Kim, Hanmin Park, Taehyun Kim, Kwanheum Cho, Eojin Lee, Soojung Ryu, Hyuk-Jae Lee, Kiyoung Choi, and Jinho Lee. 2021. GradPIM: A Practical Processing-in-DRAM Architecture for Gradient Descent. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 249–262.
- [9] Yandong Luo and Shimeng Yu. 2020. Accelerating deep neural network in-situ training with non-volatile and volatile memory based hybrid precision synapses. *IEEE Trans. Comput.* 69, 8 (2020), 1113–1127.
- [10] Tianrui Ma, Weidong Cao, Fei Qiao, Ayan Chakrabarti, and Xuan Zhang. 2022. HOGEye: Neural Approximation of HOG Feature Extraction in RRAM-Based 3D-Stacked Image Sensors. In *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*.
- [11] Peter Mattson, Christine Cheng, and et al. 2020. MLPerf Training Benchmark. arXiv:cs.LG/1910.01500
- [12] Paulius Micikevicius, Dusan Stolic, and et al. 2022. FP8 Formats for Deep Learning. arXiv:cs.LG/2209.05433
- [13] Vijay Janapa Reddi, Christine Cheng, and et al. 2020. MLPerf Inference Benchmark. arXiv:cs.LG/1911.02549
- [14] Jian-Wei Su, Yen-Chi Chou, and et al. 2021. 16.3 A 28nm 384kb 6T-SRAM Computation-in-Memory Macro with 8b Precision for AI Edge Chips. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*. 250–252.
- [15] Xiaoyu Sun, Weidong Cao, and et al. 2024. Efficient Processing of MLPerf Mobile Workloads Using Digital Compute-In-Memory Macros. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 4 (2024), 1191–1205.
- [16] Fengbin Tu and et al. 2022. A 28nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 Reconfigurable Digital CIM Processor with Unified FP/INT Pipeline and Bitwise In-Memory Booth Multiplication for Cloud Deep Learning Acceleration. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*. 1–3.
- [17] H.-S. Philip Wong, Simone Raoux, SangBum Kim, Jiale Liang, John P. Reifenberg, Bipin Rajendran, Mehdi Asheghi, and Kenneth E. Goodson. 2010. Phase Change Memory. *Proc. IEEE* 98, 12 (2010), 2201–2227.
- [18] Ping-Chun Wu, Jian-Wei Su, Yen-Lin Chung, and et al. 2022. A 28nm 1Mb Time-Domain Computing-in-Memory 6T-SRAM Macro with a 6.6ns Latency, 1241GOPS and 37.01TOPS/W for 8b-MAC Operations for Edge-AI Devices. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. 1–3.
- [19] Ping-Chun Wu, Jian-Wei Su, and et al. 2023. A 22nm 832Kb Hybrid-Domain Floating-Point SRAM In-Memory-Compute Macro with 16.2-70.2TFLOPS/W for High-Accuracy AI-Edge Devices. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. 126–128.
- [20] Jiwei Yang, Xu Shen, Jun Xing, Xinmei Tian, Houqiang Li, Bing Deng, Jianqiang Huang, and Xian-sheng Hua. 2019. Quantization Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [21] Jinshan Yue, Chaojie He, and et al. 2023. A 28nm 16.9-300TOPS/W Computing-in-Memory Processor Supporting Floating-Point NN Inference/Training with Intensive-CIM Sparse-Digital Architecture. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. 1–3.
- [22] Feichi Zhou, Zheng Zhou, Jiawei Chen, Tsz Hin Choy, Jingli Wang, Ning Zhang, Ziyuan Lin, Shimeng Yu, Jinfeng Kang, H-S Philip Wong, et al. 2019. Optoelectronic resistive random access memory for neuromorphic vision sensors. *Nature nanotechnology* 14, 8 (2019), 776–782.